SENTINELOPS.PY.

```
"""
SentinelOps: SOC Alert-Triage Module
Provides stable alert sorting, IP blocklist checking,
attack-signature matching and closest-login detection.
"""
import math
import random
import time
# ------------------------------------------------
# 1. Alert sorting
# ------------------------------------------------
def sort_alerts(alerts):
    """Sort alerts by descending severity using insertion sort."""
    alerts = list(alerts)
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
def sort_alerts_optimized(alerts):
    """Stable descending sort using Python's Timsort."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)
# ------------------------------------------------
# 2. IP blocklist
```

```
# ------------------------------------------------
def is_blocked(blocklist, ip):
    """Return True if the IP address appears in the blocklist."""
    for address in blocklist:
        if address == ip:
            return True
    return False
def prepare_blocklist(blocklist):
    """Convert a blocklist to a set for repeated lookups."""
    return set(blocklist)
def is_blocked_optimized(blocked_ips, ip):
    """Check membership in a prepared set."""
    return ip in blocked_ips
# ------------------------------------------------
# 3. Attack-signature matching
# ------------------------------------------------
def find_signature(log, pattern):
    """Find all pattern positions using brute-force matching."""
    if pattern == "":
        raise ValueError("Pattern must not be empty")
    n = len(log)
    m = len(pattern)
    hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
```

```
    return hits
def build_lps(pattern):
    """Build the KMP longest-prefix-suffix table."""
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps
def find_signature_optimized(log, pattern):
    """Find every occurrence using KMP in O(n + m) time."""
    if pattern == "":
        raise ValueError("Pattern must not be empty")
    lps = build_lps(pattern)
    hits = []
    i = 0
    j = 0
    while i < len(log):
        if log[i] == pattern[j]:
            i += 1
            j += 1
```

```
            if j == len(pattern):
                hits.append(i - j)
                j = lps[j - 1]
        elif j:
            j = lps[j - 1]
        else:
            i += 1
    return hits
# ------------------------------------------------
# 4. Closest login locations: corrected brute force
# ------------------------------------------------
def closest_logins(events):
    """Return closest event indices and Euclidean distance."""
    if len(events) < 2:
        return None, None
    best_sq = math.inf
    best_pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return best_pair, math.sqrt(best_sq)
def closest_brute_count(events):
    """Return the closest pair, distance and pair-evaluation count."""
    if len(events) < 2:
```

```
        return None, None, 0
    best_sq = math.inf
    best_pair = None
    evaluations = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            evaluations += 1
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return best_pair, math.sqrt(best_sq), evaluations
# ------------------------------------------------
# 5. Divide-and-conquer closest pair
# ------------------------------------------------
def closest_logins_optimized(events):
    """
    Find the closest pair using divide and conquer.
    Expected time complexity: O(n log n).
    Returns (pair, distance), with original event indices.
    """
    if len(events) < 2:
        return None, None
    points = [
        (x, y, index)
        for index, (x, y) in enumerate(events)
    ]
```

```
points.sort(key=lambda point: (point[0], point[1], point[2]))
def squared_distance(p, q):
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy
def solve(px):
    size = len(px)
    if size <= 3:
        best = math.inf
        pair = None
        for i in range(size):
            for j in range(i + 1, size):
                dist_sq = squared_distance(px[i], px[j])
                candidate = tuple(sorted((px[i][2], px[j][2])))
                if (
                    dist_sq < best
                    or (
                        dist_sq == best
                        and (pair is None or candidate < pair)
                    )
                ):
                    best = dist_sq
                    pair = candidate
        return best, pair
    middle = size // 2
    mid_x = px[middle][0]
    left = px[:middle]
    right = px[middle:]
    left_dist, left_pair = solve(left)
```

```
right_dist, right_pair = solve(right)
if left_dist < right_dist:
    best, pair = left_dist, left_pair
elif right_dist < left_dist:
    best, pair = right_dist, right_pair
else:
    best = left_dist
    candidates = [
        p for p in (left_pair, right_pair) if p is not None
    ]
    pair = min(candidates) if candidates else None
# Only points within the current best distance of the dividing line
# can improve the answer.
strip = [
    point for point in px
    if (point[0] - mid_x) ** 2 <= best
]
strip.sort(key=lambda point: (point[1], point[0], point[2]))
# In the planar closest-pair strip, only a constant number
# of subsequent neighbours need to be checked.
for i in range(len(strip)):
    for j in range(i + 1, min(i + 8, len(strip))):
        dy = strip[j][1] - strip[i][1]
        if dy * dy > best:
            break
        dist_sq = squared_distance(strip[i], strip[j])
        candidate = tuple(
            sorted((strip[i][2], strip[j][2]))
        )
```

```
                if (
                    dist_sq < best
                    or (
                        dist_sq == best
                        and (pair is None or candidate < pair)
                    )
                ):
                    best = dist_sq
                    pair = candidate
        return best, pair
    best_sq, pair = solve(points)
    if pair is None:
        return None, None
    return pair, math.sqrt(best_sq)
# ------------------------------------------------
# 6. Test harness
# ------------------------------------------------
def run_tests():
    """Verify all functions and required boundary conditions."""
    alerts = [
        ("A1", 3), ("A2", 9), ("A3", 5),
        ("A4", 9), ("A5", 1)
    ]
    expected_alerts = [
        ("A2", 9), ("A4", 9), ("A3", 5),
        ("A1", 3), ("A5", 1)
    ]
    assert sort_alerts(alerts) == expected_alerts
    assert sort_alerts_optimized(alerts) == expected_alerts
```

```
# Stability for equal severities.
equal_alerts = [("X", 5), ("Y", 5), ("Z", 5)]
assert sort_alerts(equal_alerts) == equal_alerts
assert sort_alerts_optimized(equal_alerts) == equal_alerts
# Empty alert list.
assert sort_alerts([]) == []
assert sort_alerts_optimized([]) == []
blocklist = [
    "10.0.0.5", "10.0.0.9", "172.16.4.2"
]
assert is_blocked(blocklist, "10.0.0.9") is True
assert is_blocked(blocklist, "10.0.0.5") is True
assert is_blocked(blocklist, "8.8.8.8") is False
assert is_blocked([], "8.8.8.8") is False
blocked_set = prepare_blocklist(blocklist)
assert is_blocked_optimized(blocked_set, "10.0.0.9") is True
assert is_blocked_optimized(blocked_set, "8.8.8.8") is False
log = "FAILFAIXFAIL"
assert find_signature(log, "FAIL") == [0, 8]
assert find_signature_optimized(log, "FAIL") == [0, 8]
# Pattern at the end.
assert find_signature("ABCXYZ", "XYZ") == [3]
assert find_signature_optimized("ABCXYZ", "XYZ") == [3]
# Pattern longer than log.
assert find_signature("ABC", "ABCDE") == []
assert find_signature_optimized("ABC", "ABCDE") == []
# Overlapping occurrences.
assert find_signature("AAAA", "AA") == [0, 1, 2]
assert find_signature_optimized("AAAA", "AA") == [0, 1, 2]
```

```
# Empty patterns are rejected consistently.
for function in (find_signature, find_signature_optimized):
    try:
        function("ABC", "")
    except ValueError:
        pass
    else:
        raise AssertionError("Empty pattern should raise ValueError")
events = [
    (2, 3), (12, 30), (40, 50),
    (5, 1), (12, 10), (3, 4)
]
pair, distance = closest_logins(events)
assert pair == (0, 5)
assert math.isclose(distance, math.sqrt(2))
pair_opt, distance_opt = closest_logins_optimized(events)
assert pair_opt == pair
assert math.isclose(distance_opt, distance)
# Fewer than two events.
assert closest_logins([]) == (None, None)
assert closest_logins([(1, 1)]) == (None, None)
assert closest_logins_optimized([]) == (None, None)
assert closest_logins_optimized([(1, 1)]) == (None, None)
# Two events.
pair, distance = closest_logins([(0, 0), (3, 4)])
assert pair == (0, 1)
assert math.isclose(distance, 5.0)
pair_opt, distance_opt = closest_logins_optimized(
    [(0, 0), (3, 4)]
```

```
    )
    assert pair_opt == (0, 1)
    assert math.isclose(distance_opt, 5.0)
    print("All functional and boundary tests passed.")
# ------------------------------------------------
# 7. Performance benchmarks
# ------------------------------------------------
def run_benchmarks():
    """Compare closest-pair distance evaluations and sort comparisons."""
    random.seed(42)
    print("\nClosest-pair benchmark:")
    for n in (1000, 4000, 8000):
        events = [
            (random.random() * 10000, random.random() * 10000)
            for _ in range(n)
        ]
        start = time.perf_counter()
        _, _, brute_evaluations = closest_brute_count(events)
        brute_time = time.perf_counter() - start
        start = time.perf_counter()
        pair_opt, distance_opt = closest_logins_optimized(events)
        optimized_time = time.perf_counter() - start
        print(
            f"n={n}: brute evaluations={brute_evaluations}, "
            f"brute time={brute_time:.4f}s, "
            f"divide-and-conquer time={optimized_time:.4f}s"
        )
        # Verify that both methods return the same minimum distance.
        pair_brute, distance_brute, _ = closest_brute_count(events)
```

```
        assert math.isclose(
            distance_brute, distance_opt, rel_tol=1e-9, abs_tol=1e-9
        )
    print("\nInsertion-sort comparison benchmark:")
    n = 1000
    sorted_alerts = [
        (f"A{i}", n - i) for i in range(n)
    ]
    # Nearly sorted input: swap two adjacent alerts.
    nearly_sorted = list(sorted_alerts)
    nearly_sorted[500], nearly_sorted[501] = (
        nearly_sorted[501], nearly_sorted[500]
    )
    random_alerts = [
        (f"A{i}", random.randint(1, n)) for i in range(n)
    ]
    _, nearly_count = insertion_sort_count(nearly_sorted)
    _, random_count = insertion_sort_count(random_alerts)
    print("Nearly sorted comparisons:", nearly_count)
    print("Random input comparisons:", random_count)
def insertion_sort_count(alerts):
    """Return insertion-sorted alerts and key-comparison count."""
    alerts = list(alerts)
    comparisons = 0
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
```

```
            if alerts[j][1] < key[1]:
                alerts[j + 1] = alerts[j]
                j -= 1
            else:
                break
        alerts[j + 1] = key
    return alerts, comparisons
if __name__ == "__main__":
    run_tests()
    run_benchmarks()
```